Implementation Strategy for Burst-Aware Adaptive Round Robin: Tools, Experimental Design and Evaluation Plan

Kunal Srivastava
24BCE2476
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

***Abstract*: This document describes how the burst-aware adaptive round robin (BARR) scheduler proposed in the first review will be implemented and evaluated. It sets out a phased implementation plan, the software and hardware tech stack, the module structure of the code base, the pipeline used to collect and label CPU bursts from Linux scheduler traces, public workload logs and synthetic generators, the design of the burst predictors and of the trace driven discrete event simulator, and the baseline schedulers against which BARR will be compared. The evaluation parameters are defined precisely, together with the experimental matrix, the ablation studies, the statistical tests and the target thresholds that will be used to judge success. Testing, validation, risks and the deliverables expected at the second review are also listed.**

***Keywords*: CPU scheduling, round robin, dynamic time quantum, burst prediction, discrete event simulation, implementation plan**

I. Introduction

The first review identified a gap between dynamic quantum round robin (RR) algorithms, which assume that the burst time of every process is known [2], [3], [4], and real operating systems, which only know the past behavior of a process. BARR closes this gap by predicting the next CPU burst of each process from its history and converting the prediction into a bounded per process quantum.

This document turns that idea into a concrete engineering and experimental plan. Its purpose is to fix, before any results are produced, what will be built, with which tools, on which data, and how the outcome will be measured, so that the comparison with the baselines is fair and repeatable. Section II gives the overall strategy and timeline, Section III the tech stack, Section IV the software architecture, Sections V to VII the data pipeline, the predictors and the simulator, Section VIII the baselines, Section IX the evaluation parameters, Section X the experimental design, Section XI testing and validation, Section XII risks, and Section XIII the deliverables.

II. Implementation Strategy

The work is organized as six phases that build on one another. The guiding principle is that the simulator and the baselines are completed and validated before any learned model is introduced. This ordering means that every later improvement can be attributed to the predictor or to the quantum policy, and not to a difference in how the schedulers are simulated.

- **Phase 1, foundation:** repository, configuration system, common trace format, discrete event simulator core with FCFS and fixed quantum RR.
- **Phase 2, data:** Linux trace collection and burst extraction, loaders for public logs, synthetic workload generators, train, validation and test splits.
- **Phase 3, baselines:** SJF with exponential averaging, SRTF oracle and the three dynamic quantum algorithms in both oracle and predicted modes; validation against published examples.
- **Phase 4, predictors:** feature builder, exponential average, gradient boosted trees and LSTM; offline accuracy and latency measurement.
- **Phase 5, BARR:** quantum controller, preemption update and aging; integration of each predictor into the simulator.
- **Phase 6, evaluation:** full experimental matrix, ablations, statistical tests, plots and the second review report.

TABLE I. Phase wise Timeline

Phase	Weeks	Exit criterion
1 Foundation	1 to 2	FCFS and RR reproduce textbook Gantt charts exactly
2 Data	2 to 4	Burst data set in Parquet with documented schema
3 Baselines	4 to 5	Dynamic RR matches the worked examples of [2] to [4]
4 Predictors	5 to 7	MAE and latency reported for all three models
5 BARR	7 to 8	BARR runs end to end on every workload
6 Evaluation	8 to 10	All tables and figures for review 2 generated by script

III. Tech Stack

Python is chosen as the main language because the simulator, the data processing and the models can then share one code base and one data format. Speed critical parts of the simulator are kept simple enough to be accelerated with Numba if profiling shows the need. Table II lists the stack.

TABLE II. Tech Stack

Layer	Tool	Use in the project
Language	Python 3.11	Simulator, data pipeline, models, analysis
Tracing	ftrace via trace-cmd; perf sched	Record sched_switch and sched_wakeup events [18]
Trace OS	Ubuntu 24.04 LTS, bare metal	Machine used to collect Linux bursts
Load	stress-ng, make -j, gzip, ffmpeg, wrk, shell scripts	Compute bound, compile, I/O mixed and interactive programs
Data	pandas, NumPy, PyArrow	Parsing, burst extraction, Parquet storage

Layer	Tool	Use in the project
Simulator	Custom discrete event engine (heapq), optional Numba	Trace driven CPU and ready queue model
Classical ML	scikit-learn [10]	Splits, metrics, linear and kNN sanity models
Boosted trees	XGBoost [8]	Gradient boosted regression predictor
Sequence model	PyTorch [11]	Small LSTM predictor [9]
Deployment check	Treelite, ONNX Runtime	Compiled inference to measure realistic latency
Config	YAML files with fixed seeds	One file per experiment, fully reproducible
Analysis	SciPy, Matplotlib, Jupyter	Statistical tests, figures, exploration
Quality	pytest, Hypothesis, Git and GitHub	Unit and property tests, version control

A virtual machine is avoided for trace collection because hypervisor scheduling adds noise to the measured run intervals. All experiments other than tracing run on an ordinary laptop, since the simulator is single threaded and the models are small.

IV. Software Architecture

The code base is split into packages that match the blocks of the architecture presented in the first review. Every scheduler, including BARR, implements one interface, so the simulator does not know which policy it is running. The planned layout is shown below.

```
barr/
  traces/      collect.sh, parse_ftrace.py,
               load_google.py, load_swf.py
  workloads/   synthetic.py (generators)
  features/    build.py (history window, stats)
  predictors/  base.py, ema.py, gbrt.py, lstm.py
  sched/       base.py, fcfs.py, sjf.py, rr.py,
               dyn_median.py, dyn_mean.py, barr.py
  sim/         engine.py, events.py, process.py
  metrics/     collector.py, fairness.py
  experiments/ configs/*.yaml, run.py
  analysis/    stats.py, plots.py
tests/         unit and property tests
```

The scheduler interface has four methods: *on_ready(p, t)*, *pick_next(t)* which returns a process and its quantum, *on_preempt(p, t)* and *on_burst_end(p, t, length)*. A predictor has *predict(history)* and *update(pid, length)*. BARR is the composition of a predictor, a quantum controller and an RR queue, which makes it easy to swap the predictor for the ablation studies.

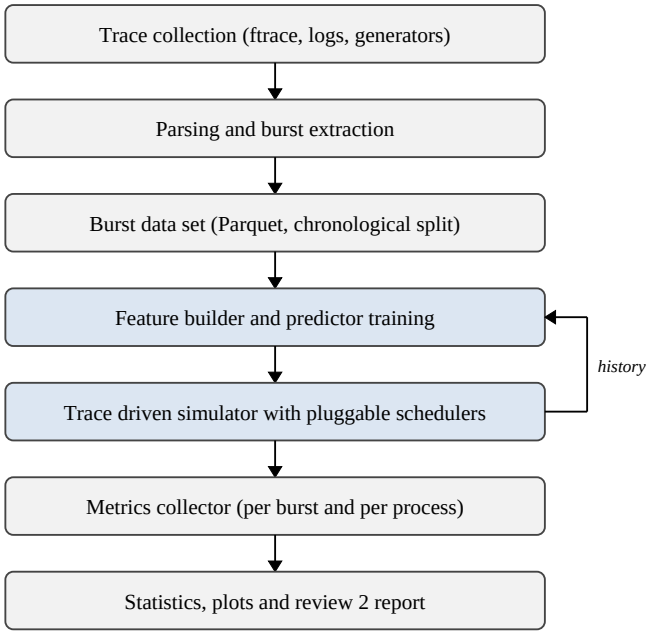


Fig. 1. Implementation pipeline. Shaded blocks contain the new components; the feedback arrow carries each measured burst back into the history used for the next prediction.

V. Data Collection and Preparation

A. Linux Scheduler Traces

Traces are recorded with trace-cmd on the *sched_switch* and *sched_wakeup* events while scripted workload mixes run for fixed periods. Each *sched_switch* record gives the outgoing thread, its state and the incoming thread, so the run interval of a thread is the time between its switch in and its next switch out.

A run interval is not always a complete CPU burst in the textbook sense. When a thread is switched out while still runnable (state R), it was preempted and its burst continues in the next interval. The parser therefore merges consecutive intervals of a thread until it is switched out in a sleeping state (S or D). The merged length is the label t_i; the time from that switch out to the next wakeup is recorded as the blocked time feature. Kernel threads and the idle task are removed.

Five workload mixes are planned: interactive (shell, editor, small scripts), compilation of a large C project, compression and media encoding, an HTTP server under wrk load, and a mixture of all four. Each mix is recorded several times so that no single run dominates the data set.

B. Public Workload Logs

Task durations from the Google cluster trace [5], [17] and job logs in the Standard Workload Format from the Parallel Workloads Archive [6] are used for heavy tailed distributions. Because these logs contain whole jobs and not bursts, the jobs of one user are ordered by submission time and treated as the burst sequence of a single pseudo process, in the spirit of [7]. Durations are rescaled by one constant per log so that their median matches the median of the Linux bursts. This mapping is a stated assumption and results on these logs will be reported separately.

C. Synthetic Workloads

Generators produce processes as alternating sequences of CPU bursts and I/O waits with Poisson arrivals. The planned families are exponential bursts, a bimodal mix of

short interactive and long compute bursts, a Pareto heavy tail, and a phase change process whose burst distribution switches part way through its life. The last case tests how quickly each predictor adapts. Offered load ρ is varied over 0.5, 0.7 and 0.9.

D. Common Format and Splits

All sources are converted to one Parquet schema: process id, source, arrival time, burst index, burst length, preceding blocked time, priority and a flag for preemption. Bursts of each process are split chronologically into 60 % training, 20 % validation and 20 % test. Hyperparameters are tuned on the validation part only, and the test part is used once for the reported numbers.

VI. Predictor Implementation

The feature vector of burst i contains the last k burst lengths (default $k = 8$, zero padded with a mask), their mean, variance, minimum and maximum, the exponential average τ, the blocked time before becoming ready, the fraction of earlier quanta that ended in preemption and the static priority. Lengths are log transformed because bursts span several orders of magnitude.

- **Exponential average:** equation (1) of the first review, with α chosen from {0.25, 0.5, 0.75} on validation data. It is also the cold start predictor for processes with fewer than k bursts.
- **Gradient boosted trees:** XGBoost with shallow trees (depth 4 to 6, at most 200 trees) so that inference stays cheap. An asymmetric loss that penalizes under prediction more heavily, following [16], will be tried because an under predicted burst costs an extra context switch.
- **LSTM:** one layer with 16 to 32 hidden units over the last 16 log burst lengths, trained with Adam and early stopping. It is treated as an upper bound on accuracy rather than as a deployable model unless its latency proves acceptable.

Each model is first evaluated offline on the test bursts. Inference latency is then measured in two ways: in Python, which is what the simulator uses, and in compiled form through Treelite for the tree model and ONNX Runtime for the LSTM, which gives a closer estimate of the cost on a real dispatch path [13], [14].

VII. Simulator Design

The simulator models one CPU and one ready queue. Time is a floating point value in milliseconds. Events are kept in a priority queue ordered by time and are of four kinds: process arrival, burst completion, quantum expiry and I/O completion. A process is a list of (CPU burst, I/O wait) pairs taken from the trace. Every dispatch of a different process than the one that ran last adds a context switch cost c, during which no useful work is done. The default is $c = 0.1$ ms, and it is swept over 0.01, 0.1 and 0.5 ms because the indirect cost of a switch through cache and TLB effects is workload dependent.

The quantum controller implements equation (3) of the first review. When a quantum expires, the predicted total burst is raised to β times the CPU time already used, with $\beta = 2$ by default, so that repeated under prediction grows the quantum geometrically up to $q_{\max}$. The aging counter of a waiting process counts how many dispatches have passed it; above a threshold A the process moves to the head of the queue. A small code sketch of the controller is shown below.

```
def quantum(p):
    pred = predictor.predict(p.history)
    if p.preempted:
        pred = max(pred, BETA * p.used)
    q = (1 + DELTA) * (pred - p.used)
    return min(Q_MAX, max(Q_MIN, q))
```

Every event is written to a log with its time, process, burst index and type. All metrics are computed afterwards from this log, so the same log can be inspected by hand when a result looks suspicious.

VIII. Baselines

All baselines run in the same simulator, with the same context switch cost and the same workloads. Table III lists them. Oracle variants are given the true burst lengths and are used only as reference points; they are not achievable by a real scheduler.

TABLE III. Baseline Schedulers

Scheduler	Burst information	Role
FCFS	None	Lower reference, no preemption
RR fixed q	None; q in {2, 5, 10, 20, 50} ms	Classical baseline; best q per workload also reported
SJF, EMA	Exponential average	Textbook practical SJF [1]
SRTF oracle	True remaining burst	Optimal mean waiting time reference
Median RR [2]	Oracle and predicted	Dynamic quantum from median
Median RR with reordering [3]	Oracle and predicted	Dynamic quantum with rearranged queue
Mean RR [4]	Oracle and predicted	Dynamic quantum from mean
BARR oracle	True burst	Upper bound of the BARR policy

In predicted mode, the remaining burst used by [2] to [4] is replaced by the predicted burst minus the CPU time already used, with a small positive floor. Running these algorithms in both modes directly answers the research question of how much of their reported gain survives without oracle knowledge.

IX. Evaluation Parameters

Let A_j, C_j, S_j and I_j be the arrival time, completion time, total CPU time and total I/O time of process j, and let T be the makespan. The scheduling metrics are then defined as follows.

$$\mathrm{TAT}_j = C_j - A_j \tag{1}$$

$$\mathrm{WT}_j = \mathrm{TAT}_j - S_j - I_j \tag{2}$$

$$\mathrm{SD}_j = \mathrm{TAT}_j / (S_j + I_j) \tag{3}$$

$$J = (\Sigma\, \mathrm{SD}_j)^2 / (n\, \Sigma\, \mathrm{SD}_j^2) \tag{4}$$

Here WT is the time spent in the ready queue, SD is the slowdown, and J is Jain's fairness index [12] over the slowdowns of the n processes, equal to 1 when all processes are slowed equally. Response time is measured per burst, from the moment a process becomes ready to its

first dispatch, which captures interactivity better than a single value per process.

TABLE IV. EVALUATION PARAMETERS

Group	Parameter	Better
Efficiency	Mean WT and mean TAT, equations (1), (2)	Lower
Interactivity	Mean and 95th percentile per burst response time	Lower
Overhead	Number of context switches; switch overhead share N_{cs} c / T	Lower
Capacity	Throughput (completed processes / T); useful CPU utilization	Higher
Fairness	Maximum WT; mean slowdown (3); Jain index (4)	Lower; lower; higher
Prediction	MAE, RMSE, share of bursts under predicted	Lower
Policy fit	Share of bursts completed within their first quantum	Higher
Cost	Median and 99th percentile inference latency; model size	Lower

The comparison with the baselines rests mainly on four of these: mean waiting time, 95th percentile response time, number of context switches and maximum waiting time. Together they show whether BARR saves switching without hurting either responsiveness or fairness. The target thresholds that will be used to call the result a success are:

- the learned predictor reduces MAE by at least 15 % relative to exponential averaging on the Linux test bursts;
- BARR needs at least 20 % fewer context switches than the best fixed quantum RR for the same workload, with 95th percentile response time within 10 % of it;
- BARR keeps maximum waiting time no worse than fixed quantum RR with $q = q_{max}$;
- compiled inference of the chosen model takes a few microseconds, small compared with a typical quantum.

These thresholds are fixed now so that the second review cannot select them after seeing the results; if they are not met, the outcome will be reported as such.

X. EXPERIMENTAL DESIGN

A. Experimental Matrix

Every scheduler of Table III and every BARR variant is run on every workload: five Linux mixes, the Google and SWF derived workloads, and four synthetic families at three loads. Synthetic workloads use ten random seeds; trace workloads use ten windows sampled from the test period.

B. Parameter Sweeps and Ablations

- **Predictor ablation:** BARR with EMA, trees, LSTM and oracle. The gap between EMA and oracle shows how much any predictor could gain; the position of the learned models inside that gap is the main result.
- **Safety margin:** δ in {0, 0.1, 0.25, 0.5}.
- **Bounds:** q_{min} in {1, 2} ms and q_{max} in {20, 50, 100} ms.
- **History length:** k in {4, 8, 16}.
- **Aging:** BARR with and without the aging rule, to show its effect on maximum waiting time.
- **Switch cost and load:** c and ρ as in Sections V and VII.

C. Statistical Treatment

Results are reported as means with 95 % bootstrap confidence intervals over seeds or windows. BARR is compared with the strongest non oracle baseline using the Wilcoxon signed rank test [15] on paired runs, at a significance level of 0.05, because the metric differences are not expected to be normally distributed. Effect sizes are reported as percentage change alongside the p values.

XI. TESTING AND VALIDATION

- **Textbook checks:** FCFS, SJF and RR are run on the worked examples of a standard text [1] and must reproduce the published Gantt charts and average waiting times exactly.
- **Reproduction of prior work:** the dynamic RR implementations are run on the process tables printed in [2] to [4] in oracle mode and must give the same quanta and averages as the papers.
- **Invariants:** property based tests with Hypothesis on random workloads check that the CPU never runs two processes at once, that total service equals the input bursts, that no time is negative and that every process completes.
- **Cross check:** fixed quantum RR is also written as a separate short loop without the event engine; both versions must agree on random inputs.
- **Data checks:** burst extraction is tested on hand built ftrace snippets that include preemptions, and summary statistics of each trace are inspected before use.
- **Leakage check:** a test confirms that no feature of a test burst uses information from a later burst.

XII. RISKS AND MITIGATION

TABLE V. RISKS

Risk	Mitigation
Tracing overhead or noise distorts burst lengths	Bare metal machine, fixed CPU frequency, repeated runs
Cluster logs do not resemble CPU bursts	Report separately; treat as stress test of heavy tails only
LSTM too slow for the dispatch path	Keep as accuracy upper bound; deploy tree model
Simulator ignores cache effects of switching	Sweep context switch cost c
Python timing misleading for kernel cost	Measure compiled models with Treelite and ONNX Runtime
Gains disappear once bursts are predicted	This is itself a result for the research question and will be reported

XIII. DELIVERABLES FOR REVIEW 2

- Source code repository with the simulator, schedulers, predictors, tests and experiment configurations.
- The labelled CPU burst data set with its collection scripts and schema description.
- Prediction accuracy and latency results for the three predictors.
- Scheduling results for all baselines and BARR on the full matrix, with ablations and statistical tests.
- A revised paper draft containing the implementation, results and discussion sections.

REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] R. J. Matarneh, "Self-adjustment time quantum in round robin algorithm depending on burst time of the now running processes," *Amer. J. Appl. Sci.*, vol. 6, no. 10, pp. 1831–1837, 2009.

[3] H. S. Behera, R. Mohanty, and D. Nayak, "A new proposed dynamic quantum with re-adjusted round robin scheduling algorithm and its performance analysis," *Int. J. Comput. Appl.*, vol. 5, no. 5, pp. 10–15, Aug. 2010.

[4] A. Noon, A. Kalakech, and S. Kadry, "A new round robin based scheduling algorithm for operating systems: Dynamic quantum using the mean average," *Int. J. Comput. Sci. Issues*, vol. 8, no. 3, pp. 224–229, May 2011.

[5] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, "Heterogeneity and dynamicity of clouds at scale: Google trace analysis," in *Proc. 3rd ACM Symp. Cloud Computing (SoCC)*, 2012, Art. no. 7.

[6] D. G. Feitelson, D. Tsafrir, and D. Krakov, "Experience with using the Parallel Workloads Archive," *J. Parallel Distrib. Comput.*, vol. 74, no. 10, pp. 2967–2982, Oct. 2014.

[7] D. Tsafrir, Y. Etsion, and D. G. Feitelson, "Backfilling using system-generated predictions rather than user runtime estimates," *IEEE Trans. Parallel Distrib. Syst.*, vol. 18, no. 6, pp. 789–803, Jun. 2007.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[12] R. Jain, D.-M. Chiu, and W. R. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corp., Tech. Rep. DEC-TR-301, 1984.

[13] J. Chen, S. S. Banerjee, Z. T. Kalbarczyk, and R. K. Iyer, "Machine learning for load balancing in the Linux kernel," in *Proc. 11th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys)*, 2020, pp. 67–74.

[14] Y. Zhang and Y. Huang, "'Learned': Operating systems," *ACM SIGOPS Oper. Syst. Rev.*, vol. 53, no. 1, pp. 40–45, Jul. 2019.

[15] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, Dec. 1945.

[16] É. Gaussier, D. Glesser, V. Reis, and D. Trystram, "Improving backfilling by using machine learning to predict running times," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2015, Art. no. 64.

[17] C. Reiss, J. Wilkes, and J. L. Hellerstein, "Google cluster-usage traces: Format + schema," Google Inc., Mountain View, CA, USA, Tech. Rep., 2011.

[18] The Linux Kernel Documentation, "ftrace: Function tracer." [Online]. Available: https://docs.kernel.org/trace/ftrace.html